Lesson5_R_Vectors

Paul Lott

2024-05-13

R Objects - Vectors

In some of the previous examples, when we split or matched patterns in strings - we got a `list` or `vector` out of the functions.

While `numeric`, `character`, `integer` are the base datatypes in R. We can combine these together to create more complex data structures.

R objects combine multiple instances of a basic datatypes to create more complex data structures.

R Datastructure	Description
vector	A vector contains an ordered collection indexed by the integers 1, 2, . . . , n, where n is the length of the vector.
factor	A factor is a special kind of vector which contains underlying numeric values 1, 2, . . . , n, but each of these n values has an associated character label (which may or may not be the numeric value).
matrix	A matrix contains a two-dimensional collection of numeric or character values indexed by pairs of integers (i, j).
array	An array contains an n-dimensional collection of numeric or character values indexed by n-tuples of integers (e.g., (i,j,k) for a 3-dimensional array).
list	A list contains an ordered collection of objects, and the objects in the list can be of different types.
data.frame	A data.frame is a type of list commonly used to store datasets, with a row for each observation and a column for each variable, and each variable can be of a different type (e.g., numeric, character, etc.).

R's Big Secret

While we've covered the Characters, Numerical and Logical datatypes. We've been ignoring an important part of R that distinguishes it from almost every other programming language.

Vectors - All of the basic datatypes are stored as a vector of length 1

```r
x = "Hello"
print(paste("Is a character a vector:", is.vector(x)))
## [1] "Is a character a vector: TRUE"
print(paste("How big of a vector:", length(x)))
## [1] "How big of a vector: 1"


y = 1.02
print(paste("Is a numerical a vector:", is.vector(y)))
## [1] "Is a numerical a vector: TRUE"
print(paste("How big of a vector:", length(y)))
## [1] "How big of a vector: 1"

z = TRUE
print(paste("Is a Logical a vector:", is.vector(z)))
## [1] "Is a Logical a vector: TRUE"
print(paste("How big of a vector:", length(z)))
## [1] "How big of a vector: 1"

a = 8L
print(paste("Is a integer a vector:", is.vector(a)))
## [1] "Is a integer a vector: TRUE"
print(paste("How big of a vector:", length(a)))
## [1] "How big of a vector: 1"

b = raw(1)
print(paste("Is a raw a vector:", is.vector(b)))
## [1] "Is a raw a vector: TRUE"
print(paste("How big of a vector:", length(b)))
## [1] "How big of a vector: 1"
```

Vector

Vectors is a collection of one datatype. In R, we refer this to *atomic* - meaning that all the elements in the `vector` are going to be stored the same `mode` in other works - the same fundamental datatype. This means that while you can define a vector with numbers and strings - it'll default to a single mode (ie. character) datatype.

We typically define vectors using `c(...)` notation. However, if you want a vector with a particular mode (ie. datatype) and length, you can create it using `vector(mode=x, length=n)`

Vectors have a very special place in R. Coming from Python, it may seem a little weird that in R almost everything is a vector. But, yes, Vectors are one of the fundamental data structures of R.

Creating a Vector

```r
x = c(1.5)
print(x)
```

```
## [1] 1.5
```

```r
print(paste("MODE:", mode(x)))
```

```
## [1] "MODE: numeric"
```

```r
print(paste("CLASS:", class(x)))
```

```
## [1] "CLASS: numeric"
```

```r
print(paste("TYPEOF:", typeof(x)))
```

```
## [1] "TYPEOF: double"
```

```r
print(paste("IS VECTOR:", is.vector(x)))
```

```
## [1] "IS VECTOR: TRUE"
```

```r
x = c(1.5, 2L)
print(x)
```

```
## [1] 1.5 2.0
```

```r
print(paste("MODE:", mode(x)))
```

```
## [1] "MODE: numeric"
```

```r
print(paste("CLASS:", class(x)))
```

```
## [1] "CLASS: numeric"
```

```r
print(paste("TYPEOF:", typeof(x)))
```

```
## [1] "TYPEOF: double"
```

```r
print(paste("IS VECTOR:", is.vector(x)))
```

```
## [1] "IS VECTOR: TRUE"
```

```r
x = c(1.5, "Hello", 2L)
print(x)
## [1] "1.5"   "Hello" "2"
print(paste("MODE:", mode(x)))
## [1] "MODE: character"
print(paste("CLASS:", class(x)))
## [1] "CLASS: character"
print(paste("TYPEOF:", typeof(x)))
## [1] "TYPEOF: character"
print(paste("IS VECTOR:", is.vector(x)))
## [1] "IS VECTOR: TRUE"
```

```r
x = c(1.5, 0.223, 2L)
print(x)
## [1] 1.500 0.223 2.000
print(paste("MODE:", mode(x)))
## [1] "MODE: numeric"
print(paste("CLASS:", class(x)))
## [1] "CLASS: numeric"
print(paste("TYPEOF:", typeof(x)))
## [1] "TYPEOF: double"
print(paste("IS VECTOR:", is.vector(x)))
## [1] "IS VECTOR: TRUE"
```

```r
x = c(TRUE, FALSE, TRUE)
print(x)
## [1]  TRUE FALSE  TRUE
print(paste("MODE:", mode(x)))
## [1] "MODE: logical"
print(paste("CLASS:", class(x)))
## [1] "CLASS: logical"
print(paste("TYPEOF:", typeof(x)))
## [1] "TYPEOF: logical"
print(paste("IS VECTOR:", is.vector(x)))
## [1] "IS VECTOR: TRUE"
```

```r
x = c(TRUE, FALSE, 'TRUE')
print(x)
```

```
## [1] "TRUE"  "FALSE" "TRUE"
```

```r
print(paste("MODE:", mode(x)))
```

```
## [1] "MODE: character"
```

```r
print(paste("CLASS:", class(x)))
```

```
## [1] "CLASS: character"
```

```r
print(paste("TYPEOF:", typeof(x)))
```

```
## [1] "TYPEOF: character"
```

```r
print(paste("IS VECTOR:", is.vector(x)))
```

```
## [1] "IS VECTOR: TRUE"
```

Creating Vectors by Sequence

In R, there are two similar functions.

Colon Operator Colon operator takes two numbers separated by a `begin:end` and creates a `vector` that starts at begin and ends at or before `end` value. It will only increment by 1, so if the ending value is less than one from the last value, it'll be excluded.

If the beginning value is more than then ending value, then the values are incremented by -1.

```
x = 1:20
print(x)
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
```

```
(x = 1:19.9)
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
```

```
x = 1.1:10
print(x)
```

```
## [1] 1.1 2.1 3.1 4.1 5.1 6.1 7.1 8.1 9.1
```

```
x = 10:1
print(x)
```

```
##  [1] 10  9  8  7  6  5  4  3  2  1
```

```
x = -1:2
print(x)
```

```
## [1] -1  0  1  2
```

Sequence Operator The sequence function `seq(from, to, by, length.out=NULL, along.with=NULL)`. Allows you to generate a sequence that increments by any value.

```
print(seq(1,20))
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
```

```
print(seq(1,10,by=0.1))
```

```
##  [1]  1.0  1.1  1.2  1.3  1.4  1.5  1.6  1.7  1.8  1.9  2.0  2.1  2.2  2.3  2.4
## [16]  2.5  2.6  2.7  2.8  2.9  3.0  3.1  3.2  3.3  3.4  3.5  3.6  3.7  3.8  3.9
## [31]  4.0  4.1  4.2  4.3  4.4  4.5  4.6  4.7  4.8  4.9  5.0  5.1  5.2  5.3  5.4
## [46]  5.5  5.6  5.7  5.8  5.9  6.0  6.1  6.2  6.3  6.4  6.5  6.6  6.7  6.8  6.9
## [61]  7.0  7.1  7.2  7.3  7.4  7.5  7.6  7.7  7.8  7.9  8.0  8.1  8.2  8.3  8.4
## [76]  8.5  8.6  8.7  8.8  8.9  9.0  9.1  9.2  9.3  9.4  9.5  9.6  9.7  9.8  9.9
## [91] 10.0
```

```r
print(seq(1,10,length.out=25))
```

```
##  [1]  1.000  1.375  1.750  2.125  2.500  2.875  3.250  3.625  4.000  4.375
## [11]  4.750  5.125  5.500  5.875  6.250  6.625  7.000  7.375  7.750  8.125
## [21]  8.500  8.875  9.250  9.625 10.000
```

Indexing Vectors

Vectors can be indexed using the `[]`

Note: **R is 1-based.** The first element is at position 1

You can use the `:` operator, or `seq` function, or other comparison operators to subset vectors.

```r
(x = seq(1,10,length.out=25))
```

```
##  [1]  1.000  1.375  1.750  2.125  2.500  2.875  3.250  3.625  4.000  4.375
## [11]  4.750  5.125  5.500  5.875  6.250  6.625  7.000  7.375  7.750  8.125
## [21]  8.500  8.875  9.250  9.625 10.000
```

```r
print(x[15])
```

```
## [1] 6.25
```

```r
print(x[2]*3)
```

```
## [1] 4.125
```

```r
print(x[1:10])
```

```
##  [1] 1.000 1.375 1.750 2.125 2.500 2.875 3.250 3.625 4.000 4.375
```

```r
print(x[c(1,3,5,25)])
```

```
## [1]  1.00  1.75  2.50 10.00
```

```r
print(x[x>1.5 & x<3.75])
```

```
## [1] 1.750 2.125 2.500 2.875 3.250 3.625
```

Negative indices in R are not considered from the reverse end of the vector - but rather negative values denote indices that should be excluded.

```r
x = seq(1,10)

print(x[-(2:5)])
```

```
## [1]  1  6  7  8  9 10
```

Vector Functions

We've seen functions and operators in R, for Character strings and numerical datatypes.

What about all the previous functions?

They still work! They'll just work on a vector of length 1 or a vector of length 100.

However, it's important to understand how they work when you do math with a scalar vs multiple vectors

Scalar math on a Vector

If you are performing function on a single vector - it's as expected. R will apply the function to each element in the vector.

```
x = 1:10
print(x+12)
```

```
##  [1] 13 14 15 16 17 18 19 20 21 22
```

```
print(x*2)
```

```
##  [1]  2  4  6  8 10 12 14 16 18 20
```

```
print(cos(x))
```

```
##  [1]  0.5403023 -0.4161468 -0.9899925 -0.6536436  0.2836622  0.9601703
##  [7]  0.7539023 -0.1455000 -0.9111303 -0.8390715
```

```
print(x^2)
```

```
##  [1]   1   4   9  16  25  36  49  64  81 100
```

Vector-Vector Math

If you use two vectors of the same length. You can think of the value of each vector being applied to the corresponding value in the other vector

Vector 1	1	2	3	4	5	6	7	8	9	10
Vector 2	11	12	13	14	15	16	17	18	19	20
Addition	12	14	16	18	20	22	24	26	28	30

```
x = 1:10
y = 11:20

print(x+y)
```

```
##  [1] 12 14 16 18 20 22 24 26 28 30
```

If you use two vectors of differing size, then R will duplicate the shorter of the 2 until it's the same length as the longer vector. However, when doing numerical math if the shorter vector is not multiple of the length of the longer vector - it'll warn you and produce an error.

Vector 1 = c(1,2,3,4,5,6,7,8,9,10,11,12)

Vector 2 = c(11,12,13)

Vector 1	1	2	3	4	5	6	7	8	9	10	11	12
Vector 2	11	12	13	11	12	13	11	12	13	11	12	13
Addition	12	14	16	15	17	19	18	20	22	21	23	25

```r
x = 1:12
y = c(0.1, 0.2, 0.29, 1, 4)

print(x)
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12
```

```r
print(y)
```

```
## [1] 0.10 0.20 0.29 1.00 4.00
```

```r
print(x+y)
```

```
## Warning in x + y: longer object length is not a multiple of shorter object
## length

##  [1]  1.10  2.20  3.29  5.00  9.00  6.10  7.20  8.29 10.00 14.00 11.10 12.20
```

With character string vectors of unequal length, functions such at `paste()`- notice what happens when we `paste()` the two vectors together.

```r
x = c("one", "two")
y = c(1,2,3,4,5)
paste(x,y)
```

```
## [1] "one 1" "two 2" "one 3" "two 4" "one 5"
```

The shorter vector is repeated, but if it doesn't exactly match up then those values are excluded.

Subsetting Vector

Selecting element from the vector is called Subsetting. The syntax is shared for addition R datastructures (List, Matrices, Dataframes, Arrays).

Selecting multiple elements

Use `[]` to select any number of elements from a vector. To illustrate, I'll apply `[]` to 1D atomic vectors, and then in the later section about more complex objects and more dimensions we'll go over how it works with those.

Let's explore the different types of subsetting with a simple vector, `x`.

```r
x <- c(2.1, 4.2, 3.3, 5.4, 1.5, 6.6, 9.7)
```

Note that the number after the decimal point represents the original position in the vector.

There are six things that you can use to subset a vector:

1. **Positive integers** return elements at the specified positions:

   ```r
   x[c(3, 1)]
   ```

   ```
   ## [1] 3.3 2.1
   ```

   Duplicate indices will duplicate values

   ```r
   x[c(1, 1)]
   ```

   ```
   ## [1] 2.1 2.1
   ```

   Real numbers are silently truncated to integers

   ```r
   x[c(2.1, 2.9)]
   ```

   ```
   ## [1] 4.2 4.2
   ```

2. **Negative integers** exclude elements at the specified positions:

   ```r
   x[-c(3, 1)]
   ```

   ```
   ## [1] 4.2 5.4 1.5 6.6 9.7
   ```

   Note that you can't mix positive and negative integers in a single subset:

   ```r
   x[c(-1, 2)]
   ```

   ```
   ## Error in x[c(-1, 2)]: only 0's may be mixed with negative subscripts
   ```

3. **Logical vectors** select elements where the corresponding logical value is `TRUE`. This is probably the most useful type of subsetting because you can write an expression that uses a logical vector:

   ```r
   print(x)
   ```

   ```
   ## [1] 2.1 4.2 3.3 5.4 1.5 6.6 9.7
   ```

   ```r
   x[c(TRUE, TRUE, FALSE, FALSE)]
   ```

   ```
   ## [1] 2.1 4.2 1.5 6.6
   ```

```r
x[x > 3]
```

```
## [1] 4.2 3.3 5.4 6.6 9.7
```

In `x[y]`, what happens if `x` and `y` are different lengths? The behaviour is controlled by the **recycling rules** where the shorter of the two is recycled to the length of the longer. This is convenient and easy to understand when one of `x` and `y` is length one, but I recommend avoiding recycling for other lengths because the rules are inconsistently applied throughout base R.

```r
x[c(TRUE, FALSE)]  #Note differences in vector lengths x and logical (R repeats logical vector)
```

```
## [1] 2.1 3.3 1.5 9.7
```

Note that a missing value in the index always yields a missing value in the output:

```r
print(x)
```

```
## [1] 2.1 4.2 3.3 5.4 1.5 6.6 9.7
```

```r
x[c(TRUE, TRUE, NA, FALSE)]
```

```
## [1] 2.1 4.2  NA 1.5 6.6  NA
```

4. **Nothing** returns the original vector. This is not useful for 1D vectors, but, as you'll see shortly, is very useful for matrices, data frames, and arrays. It can also be useful in conjunction with assignment.

```r
x[]
```

```
## [1] 2.1 4.2 3.3 5.4 1.5 6.6 9.7
```

5. **Zero** returns a zero-length vector. This is not something you usually do on purpose, but it can be helpful for generating test data.

```r
x[0]
```

```
## numeric(0)
```

Additional Vector Functions

Vector Function	Description
length(x)	Return length of vector
sort(x)	Sort or order a vector
order(x)	Returns vector of indices of sorted values
unique(x)	Returns only unique items
rep(x, . . .)	Replicate Elements of Vectors
union(x,y)	Returns a union of x and y
intersect(x,y)	Returns the intersection of x and y

```r
x = c(1,2,3,4)
print(length(x))
```

```
## [1] 4
```

```r
x = c(3, 2, 1, 4, 10, 23, 1, 22, 2)
print(sort(x))
```

```
## [1]  1  1  2  2  3  4 10 22 23
```

```r
print(order(x))
```

```
## [1] 3 7 2 9 1 4 5 8 6
```

```r
print(unique(x))
```

```
## [1]  3  2  1  4 10 23 22
```

Changing the Length of a Vector

Vectors also have some unique behavior when we want to change the length of the vector.

However, in R - it just extends the list and will insert **NA** in the positions where the vector was previously undefined.

```r
x = c(1)
x[5] = 210
print(x)
```

```
## [1]   1  NA  NA  NA 210
```

To shorten a vector, you can also just assign a different value to the `length(x)` function.

```r
x = c(1, NA, NA, NA, 210)
length(x) = 2
print(x)
```

```
## [1]  1 NA
```

- Start 4/29/2026

Element-wise Logical AND/OR Operators for Vectors

Remember, R loves vectors. Element-wise AND Operator `&` evaluates for each item and returns a logical vector. Whereas, Logical AND operator only returns a single `TRUE` or `FALSE`

If we use `&` of `|` - we may get a logical vector back - because the AND/OR is evaluated for each item.

```r
c(TRUE, TRUE) & c(TRUE, FALSE)
```

```
## [1]  TRUE FALSE
```

Whereas, if we are using the `&&` or `||` - we'll get an error because `&&` or `||` only want to return a single `TRUE` or `FALSE`

```r
c(TRUE, TRUE) && c(TRUE, FALSE)
```

```
## Error in c(TRUE, TRUE) && c(TRUE, FALSE): 'length = 2' in coercion to 'logical(1)'
```

```r
c(TRUE, TRUE, FALSE, FALSE) & c(TRUE, FALSE, TRUE, FALSE)
```

```
## [1]  TRUE FALSE FALSE FALSE
```

Any or All

`any()` is a function that works on a logical vector and will return `TRUE` if any values are `TRUE` and `FALSE` if none are.

`all()` is the complement function to `any()`. It will only return `TRUE` if all the values are `TRUE`.

`{r} any(c(TRUE, TRUE) & c(TRUE, FALSE))}`

`{r} all(c(TRUE, TRUE) & c(TRUE, FALSE))}`

%in% Operator

Checks if an element belongs is in the vector and returns a logical vector, where TRUE represents the position where the value are in the vector

```r
x = c(1,2,3,4,5,6,7,8)
y = c(2,4,6,8,10,12,14,16)

x %in% y
```

```
## [1] FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE
```

```r
(subset = x[x %in% y])  # Return the subset of values from X that are in Y
```

```
## [1] 2 4 6 8
```

Named Vectors

That's because these aren't base data structures in R. In R, we can use a `vector` similarly to a dictionary, were we assign keys as `names` and then the value of those `names`. **But the names don't have to be unique.** In this way, Named vectors are different Dictionaries or Hashes.

Creating a Named Vector

```
x = c("Lines"=10, "Yards"=2)
print(x)
```

```
## Lines Yards
##    10     2
```

or

In two steps, we first create the vector and then associate names/keys with the values in the vector

```
x = c(10, 2, 3)
keys = c("Lines", "Yards", "Lines")
names(x) = keys

print(x)
```

```
## Lines Yards Lines
##    10     2     3
```

Notice that we have two names that are the same.

Accessing the Value

To get the value from the array, we can then use the name/key.

We can use the indexing operator `[]` to get the name and value by indices. Or we can use the name like we did in Python.

```
x = c("Lines"=10, "Yards"=2)

print(x[2])
```

```
## Yards
##     2
```

```
print(x['Yards'])
```

```
## Yards
##     2
```

You'll notice that it's returning both the `Name` and `Value`. If we want to access only the value, we need to use `[[]]` double square brackets.

```
x = c("Lines"=10, "Yards"=2, "Yards"=4)
print(x)
```

```
## Lines Yards Yards
##    10     2     4
```

```r
print(x[[2]])
```

```
## [1] 2
```

```r
print(x[['Yards']])
```

```
## [1] 2
```

Note: When using Named Vectors, you're still limited by the fact that `vectors` are atomic types. Meaning you can't mix different datatypes as `values`. To use different datatypes, see `Named Lists` or `hash`

Subsetting Named Vectors

6. If the vector is named, you can also use **character vectors** to return elements with matching names.

```r
x = c(1, 2, 3, 4)
(y <- setNames(x, c('a','b','c', 'd')))
```

```
## a b c d
## 1 2 3 4
```

Accessing value from names

```r
y[c("d", "c", "a")]
```

```
## d c a
## 4 3 1
```

Like integer indices, you can repeat indices

```r
(y[c("a", "a", "a")])
```

```
## a a a
## 1 1 1
```

When subsetting with [, names are always matched exactly

```r
z <- c(abc = 1, def = 2)

(z[c("abd", "def")])
```

```
## <NA>  def
##   NA    2
```

Factors

A **factor** is a vector used to specify a discrete classification (category, grouping, or enumerated type) of the components of other vectors. We typically use Factors in model formula. Case vs Control, or different Categorical data.

Factors by default are considered to be ordered. However, R does support unordered **factors**

The category/group is defined as the `level` of the factor. We can also add separate `labels` that correspond to each `level`

To Create a Factor

We can create a factor by starting with a vector and passing it to `factor()`

```
x = c('Fruit', 'Vegetable', 'Fruit', 'Vegetable', 'Mineral')
(y = factor(x))
```

```
## [1] Fruit     Vegetable Fruit     Vegetable Mineral
## Levels: Fruit Mineral Vegetable
```

You'll notice that the vector of 5 elements, is part of the factor, but it has an added attribute called `levels`

The `levels` vector contains a unique set of categories and we can get the levels of the factor by `levels()` function.

```
levels(y)
```

```
## [1] "Fruit"     "Mineral"   "Vegetable"
```

The `factor` creation function also has additional parameters.

```
# Using Labels - Maps multiple levels to two groups

x <- c("Man", "Male", "Man", "Lady", "Female")
xf <- factor(x, levels = c("Male", "Man" , "Lady",   "Female"),
                labels = c("Male", "Male", "Female", "Female"))
print(xf)
```

```
## [1] Male   Male   Male   Female Female
## Levels: Male Female
```

Utilizing Factors to calculate Group specific information

The `tapply` function allows you to apply a R function across a value vector that corresponds to a the factor array.

For example, If we have a vector of number of bushels that corresponds to the factor. We could apply the `sum()` function to get a list of total bushels per category or group.

```r
bushel = c(1, 3, 1, 2, 1)
x = c('Fruit', 'Vegetable', 'Fruit', 'Vegetable', 'Mineral')
y = factor(x)
print(tapply(bushel, y, sum))
```

```
##     Fruit   Mineral Vegetable
##         2         1         5
```

Ordered Factors

The levels of factors are stored in alphabetical order, or in the order they were specified to `factor` if they were specified explicitly.

Sometimes the levels will have a natural ordering that we want to record and want our statistical analysis to make use of. The `ordered()` function creates such ordered factors but is otherwise identical to factor. For most purposes the only difference between ordered and unordered factors is that the former are printed showing the ordering of the levels, but the contrasts generated for them in fitting linear models are different.

Reading and Writing Vectors

You can write a vector using the `write()` function and read using the `scan()` function.

Syntax:

```
x = scan(file="", what=<typeof>, sep=" ") # Input
write(x, file="", sep=" ")
```

The `what` argument should be the typeof data: `integer()`, `double()`, `character()`, `logical()`

```r
x = c(1, 1, 2, 34, 2.8, 2008, 2E20)
write(x, file='numeric_vector.txt')

x = scan('numeric_vector.txt', what=double())
x
```

```
## [1] 1.000e+00 1.000e+00 2.000e+00 3.400e+01 2.800e+00 2.008e+03 2.000e+20
```